

Multi-Agent Real-Time Delivery Optimization on Grid-Based Maps: A Comparative Study of Heuristic Solvers

Le Duc Quang

University of Engineering and Technology, Vietnam National University

21021530@vnu.edu.vn - <https://www.kaggle.com/code/ducquangle1208/graph-shopee-v1> - <https://github.com/ducquangle1208/Graph>

Abstract

This paper presents a multi-agent delivery optimization system operating in a real-time, online environment on an $N \times N$ grid map. We formulate the problem as a multi-agent pickup-and-delivery (MAPD) task with stochastic order arrivals, weight/capacity constraints, and priority-differentiated reward functions. Four algorithmic approaches are designed and evaluated: Greedy BFS (baseline), VRP-style rolling horizon heuristic, ACO-inspired online solver, and a lightweight MAPD-CBS with prioritized conflict resolution. Experimental results across six benchmark configurations demonstrate that the ACO-inspired solver achieves the highest total net reward (3,635.02 points), outperforming the Greedy BFS baseline by 22.3%, while the MAPD-CBS solver excels in dense multi-shipper scenarios with frequent spatial conflicts. All solvers operate within a 8.3-second total wall-clock time budget across all tests.

Keywords—Multi-agent systems; pickup-and-delivery; online optimization; greedy BFS; ant colony optimization; conflict-based search; vehicle routing.

I. INTRODUCTION

Last-mile delivery logistics represent a critical bottleneck in modern supply chains. The advent of multi-robot and multi-courier systems has brought renewed attention to the Multi-Agent Pickup and Delivery (MAPD) problem, in which multiple agents must cooperatively service a stream of dynamically appearing orders within time and capacity constraints [1].

In this work, we address a discrete-time, online variant of MAPD on an $N \times N$ grid with static obstacles. Orders materialize stochastically over time, carrying weight, priority class, and hard/soft deadlines. Each shipper agent must decide at each timestep whether to move, pickup, or deliver, subject to weight capacity and slot capacity constraints. The objective is to maximize total net reward over T timesteps.

Our key contributions are: (i) a unified environment and scoring framework; (ii) four solver implementations with shared BFS caching infrastructure; (iii) an online heuristic scoring function that accounts for expected reward, urgency, distance, and lateness; and (iv) a comparative empirical study across six heterogeneous benchmark configurations.

II. PROBLEM FORMULATION

A. Environment Model

The environment is an $N \times N$ grid where each cell is either free (0) or occupied by a static obstacle (1). At each discrete timestep $t \in \{1, \dots, T\}$, each shipper i executes an action (move, cargo_op) where:

move $\in \{S, L, R, U, D\}$
cargo_op $\in \{0: \text{none}, 1: \text{pickup}, 2: \text{deliver}\}$

The per-step processing order is: Move $\rightarrow$ Pickup $\rightarrow$ Deliver $\rightarrow$ Reveal new orders. Collision priority is given to the shipper with the smallest id.

B. Order Structure

Each order g_i is a 7-tuple:

$g_i = \langle sx_i, sy_i, ex_i, ey_i, et_i, w_i, p_i \rangle$

where (sx_i, sy_i) is the pickup location, (ex_i, ey_i) is the delivery destination, et_i is the deadline, $w_i \in \mathbb{R}^+$ is the weight, and $p_i \in \{1, 2, 3\}$ encodes the priority class (standard, fast, express).

C. Constraints

Each shipper i maintains a bag of held orders, subject to:

$\sum w_j \leq W_{\max}(i)$ (weight capacity)
 $|bag(i)| \leq K_{\max}(i)$ (slot capacity)

Shippers may not exit the grid or enter obstacle cells.

D. Reward Function

Base reward r_{base} is determined by order weight class (Table I). On-time delivery yields:

$$r(i) = \alpha_p \times r_{\text{base}}(i) \times (1 + \text{bonus})$$

$$\text{bonus} = \max(0, (\text{et}_i - t_{\text{del}}) / \text{et}_i)$$

Late delivery yields a penalized reward:

$$r(i) = \beta_p \times r_{\text{base}}(i) \times \max(0, 1 - (t_{\text{del}} - \text{et}_i) / T)$$

Movement cost is:

$$rc(i, t) = -0.01 \times (1 + W_{\text{carried}}(i, t) / W_{\text{max}}(i))$$

This cost applies only when the shipper moves (not when stationary).

Table I: Weight-based reward parameters

Weight Range (kg)	f_weight	r_base
$w \leq 0.2$	0.4	4
$0.2 < w \leq 3$	1.0	10
$3 < w \leq 10$	1.5	15
$10 < w \leq 30$	2.0	20
$w > 30$	3.0	30

Table II: Priority multipliers

Priority	Type	α_p (on-time)	β_p (late)
3	Express	3.0	0.5
2	Fast	2.0	0.3
1	Standard	1.0	0.1

III. SYSTEM ARCHITECTURE

A. Software Structure

The codebase is organized into a core simulation environment and a pluggable solver layer:

- `env.py`: Grid environment, step logic, and scoring engine.
- `solvers/solver.py`: Abstract base class for all solvers.
- `solvers/common.py`: Shared BFS, distance cache, and heuristic scoring.
- `solvers/{greedy_bfs, vrp_or-tools, aco_solver, mapd_cbs_solver}.py`: Algorithm-specific decision logic.

The grader (`run_test.py`) evaluates all solvers against six configurations defined in `test_config.txt`. All code operates offline—no external libraries beyond the Python standard library are required.

B. Shared BFS Infrastructure

Since Manhattan distance is inaccurate in the presence of obstacles, all solvers rely on true shortest-path distances computed via BFS on the grid. Two LRU-style caches are maintained:

`_distance_cache[(start, goal)]` → path length
`_next_move_cache[(start, goal)]` → first action

These caches are effective for small maps ($N \leq 20$) and significantly reduce redundant computation across timesteps.

C. Online Decision Loop

All solvers follow the same online control loop:

```
obs = env.reset()
while not obs['done']:
    actions = solver.decide(obs)
    obs, reward, done, info = env.step(actions)
```

At each timestep, a solver: (1) reads the current observation; (2) evaluates visible orders using the heuristic scoring function; (3) assigns pickup or delivery targets to shippers; and (4) returns BFS-guided movement actions.

IV. ALGORITHMIC APPROACHES

A. Greedy BFS (Baseline)

Greedy BFS is the simplest solver. At each timestep, each shipper independently selects the highest-scoring visible order for pickup (or the most urgent held order for delivery) and moves toward it via BFS. A global reservation set prevents two shippers from targeting the same pickup order simultaneously.

The heuristic score integrates expected reward, priority bonus, urgency bonus, distance penalty, and lateness penalty:

$$\text{score} = \text{expected_reward} + \text{priority_bonus} + \text{urgency_bonus} - \text{distance_penalty} - \text{lateness_penalty}$$

Complexity: $O(T \times C \times O_{\text{vis}} \times N^2)$, where T is the horizon, C is the number of shippers, O_{vis} is the number of visible orders, and N^2 represents BFS on the grid.

B. VRP-Style Rolling Horizon

The VRP-style solver treats all currently visible orders as a small dynamic Vehicle Routing Problem and approximates local routing decisions for each shipper. Rather than relying on external OR-Tools, the solver uses a weighted scoring heuristic with tuned parameters:

$$\begin{aligned} \text{pickup_weight} &= 1.43, \text{ delivery_weight} = 0.93 \\ \text{priority_weight} &= 3.77, \text{ urgency_weight} = 4.76 \\ \text{distance_weight} &= 0.63, \text{ lateness_weight} = 2.56 \end{aligned}$$

The solver replans every timestep, making it fully online and robust to new order arrivals.

C. ACO-Inspired Solver

The ACO-inspired solver borrows the attractiveness concept from Ant Colony Optimization. Orders with high reward, high priority, and imminent deadlines are assigned high attractiveness; long travel distances and expected late deliveries are penalized. The tuned parameter set emphasizes priority and urgency:

$$\begin{aligned} \text{pickup_weight} &= 1.345, \text{ delivery_weight} = 0.888 \\ \text{priority_weight} &= 10.44, \text{ urgency_weight} = 9.82 \\ \text{distance_weight} &= 0.943, \text{ lateness_weight} = 1.92 \end{aligned}$$

This solver achieves the highest total net reward across Phase 1 benchmarks by aggressively prioritizing high-value orders even at the cost of more late deliveries.

D. MAPD-CBS Lightweight

The MAPD-CBS solver addresses the multi-agent coordination dimension by implementing a lightweight prioritized conflict resolution scheme. Shippers are processed in ascending id order; each shipper reserves its next cell, and subsequent shippers must yield if their planned move collides with a reserved cell.

This is a single-level approximation of the full Conflict-Based Search (CBS) algorithm [2], adapted for the online setting where a full constraint tree would be computationally prohibitive. Key parameters:

$$\begin{aligned} \text{pickup_weight} &= 1.36, \text{ delivery_weight} = 0.87 \\ \text{priority_weight} &= 5.0, \text{ urgency_weight} = 6.65 \\ \text{distance_weight} &= 1.16, \text{ lateness_weight} = 2.65 \\ \text{reserve_next_cells} &= \text{True} \end{aligned}$$

V. EXPERIMENTAL EVALUATION

A. Benchmark Configurations

Six configurations (C1–C6) are defined in test_config.txt, spanning varying grid sizes, shipper counts, order volumes, and time horizons. All experiments were run on a standard development machine; total wall-clock time across all 24 solver-configuration pairs was 8.3 seconds (budget: 3,600 seconds).

B. Aggregate Results

Table III: Total net reward per solver (Phase 1)

Solver	Total Net Reward	vs. Greedy BFS
ACO	3,635.02	+22.3%

Solver	Total Net Reward	vs. Greedy BFS
MAPD-CBS	3,019.72	+1.6%
Greedy BFS	2,971.82	—
VRP-OrTools	2,784.42	−6.3%

C. Per-Configuration Results

Table IV: Detailed results per configuration

Config	Method	Net Reward	% Delivered	% On-Time	Time (s)
C1	GreedyBFS	266.74	86.7%	100.0%	0.0
C1	VRP	266.74	86.7%	100.0%	0.0
C1	ACO	266.74	86.7%	100.0%	0.0
C1	MAPD-CBS	266.74	86.7%	100.0%	0.0
C2	GreedyBFS	303.77	72.0%	94.4%	0.0
C2	VRP	319.06	76.0%	89.5%	0.0
C2	ACO	409.57	96.0%	75.0%	0.0
C2	MAPD-CBS	381.59	96.0%	79.2%	0.0
C3	GreedyBFS	274.49	17.5%	100.0%	0.1
C3	VRP	548.02	72.5%	72.4%	0.1
C3	ACO	878.20	92.5%	91.9%	0.1
C3	MAPD-CBS	801.49	85.0%	91.2%	0.1
C4	GreedyBFS	436.21	35.0%	90.5%	0.3
C4	VRP	937.88	88.3%	86.8%	0.3
C4	ACO	672.56	58.3%	85.7%	0.3
C4	MAPD-CBS	181.77	16.7%	80.0%	0.3
C5	GreedyBFS	421.30	28.8%	78.3%	0.7
C5	VRP	239.11	25.0%	60.0%	0.6
C5	ACO	334.76	28.8%	65.2%	0.7
C5	MAPD-CBS	651.22	53.8%	67.4%	0.7
C6	GreedyBFS	1269.30	66.0%	89.4%	1.0
C6	VRP	473.62	22.0%	86.4%	0.6
C6	ACO	1073.19	69.0%	72.5%	1.1
C6	MAPD-CBS	736.92	43.0%	69.8%	1.1

VI. DISCUSSION

A. ACO-Inspired Solver

The ACO solver dominates across C1–C3, configurations with moderate order volumes and large reward variance across priority classes. By aggressively weighting priority (10.44) and urgency (9.82), the solver effectively filters out low-value orders in favor of express and fast deliveries. However, in C4, the ACO solver scores lower than VRP (672.56 vs. 937.88), suggesting that at higher density, its greediness on high-priority orders can leave many orders undelivered.

B. VRP-Style Solver

The VRP solver's routing-oriented design excels in C4, where capacity management and multi-stop routing are critical. Its weaker performance on C6 (473.62 vs. 1269.30 for Greedy BFS) indicates sensitivity to configuration-specific dynamics—possibly due to its tendency to over-commit to planned routes as the problem scale increases.

C. MAPD-CBS Solver

The MAPD-CBS solver is the only approach to outperform Greedy BFS in C5 (651.22 vs. 421.30), a configuration likely characterized by bottleneck map topology where inter-shipper conflicts are frequent. The lightweight prioritized conflict resolution prevents deadlocks that degrade all other solvers in narrow-corridor environments. Its weakness in C4 (181.77) suggests that the strict priority ordering disadvantages high-id shippers in large fleets.

D. Greedy BFS Baseline

Greedy BFS achieves the highest net reward in C6 (1,269.30), the largest configuration. This counterintuitive result suggests that at high scale, the additional complexity of the VRP and ACO solvers introduces suboptimal decisions, while the simple, fast per-step replanning of Greedy BFS benefits from its inherent adaptability.

E. Trade-Off Summary

Table V: Qualitative trade-off analysis

Solver	Strengths	Weaknesses
Greedy BFS	Simple, stable, scalable	Local optima, no long-horizon planning
VRP-Style	Multi-order routing efficiency	Sensitive to scale; no true VRP optimality
ACO	High-value order prioritization	May sacrifice delivery rate for reward
MAPD-CBS	Conflict resolution in bottlenecks	High-id shipper starvation in large fleets

VII. ADAPTIVE STRATEGY FOR SURGE AND HOTSPOT CONDITIONS

Phase 1 does not disclose surge zones or hotspot locations in advance. All four solvers handle these conditions through the same online replanning mechanism: at each timestep, the solver reads the current observation and scores all visible orders. When orders cluster spatially (a hotspot), the shared heuristic naturally assigns lower distance penalties to nearby orders, causing shippers already in the area to preferentially pick them up.

Urgency and priority bonuses ensure that express orders generated during a surge are served first. Shippers with remaining capacity in the vicinity will absorb additional orders organically. No precomputed hotspot maps or forecast models are required, making the approach fully distribution-agnostic.

VIII. CONCLUSION

We have presented four heuristic solvers for online multi-agent delivery optimization on grid maps. The ACO-inspired solver achieves the best aggregate performance (3,635.02 total net reward), demonstrating the value of priority- and urgency-weighted order selection. The MAPD-CBS solver provides the best conflict resolution in spatially constrained environments. Greedy BFS remains surprisingly competitive at large scale, and the VRP-style solver excels in medium-density, high-capacity configurations.

Future work will investigate learned heuristics (e.g., via reinforcement learning), more complete CBS implementations with look-ahead, and dynamic weight adaptation based on observed order distributions.

REFERENCES

- [1] M. Liu, H. Ma, J. Li, and S. Koenig, "Task and Path Planning for Multi-Agent Pickup and Delivery," in *Proc. AAMAS 2019*, pp. 1152–1160.
- [2] G. Sharon, R. Stern, A. Felner, and N. Sturtevant, "Conflict-Based Search for Optimal Multi-Agent Pathfinding," *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.
- [3] M. Dorigo and T. Stützle, *Ant Colony Optimization*. MIT Press, 2004.
- [4] P. Toth and D. Vigo, Eds., *The Vehicle Routing Problem*. SIAM, 2002.
- [5] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.